UTM
UNIVERSITI TEKNOLOGI MALAYSIA

UNIVERSITI TEKNOLOGI MALAYSIA
ASCEND 2030

Current Trends and Ethics

Week 1 • Special Topics in Java Web Application



Introduction

Explore current Java web technologies and evaluate responsible development choices. This week connects the web application stack with ethical use of AI, data and shared code.

Week 1 Learning Goals

01 Explain how Java web applications exchange HTTP messages.

02 Describe the evolution from Servlets and JSP to Spring-based applications.

03 Evaluate AI-assisted coding choices using ethical and technical criteria.

Static and dynamic web pages

01 **Static**: the server returns an existing file, such as an information page.

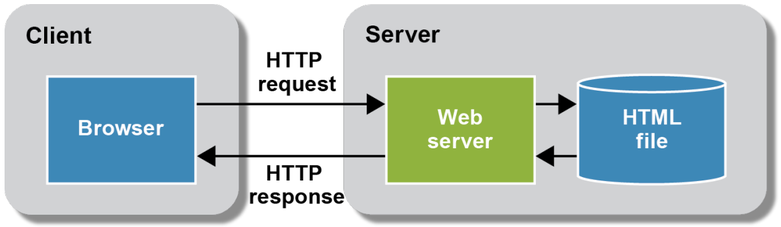
02 **Dynamic**: the application produces content using request or database data.

03 Java code executes on the server. JavaScript can execute in the browser.

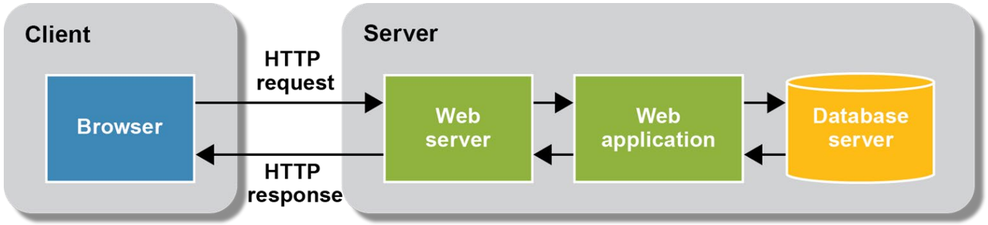
04 Both approaches return content that the browser can display.

Static and dynamic web pages

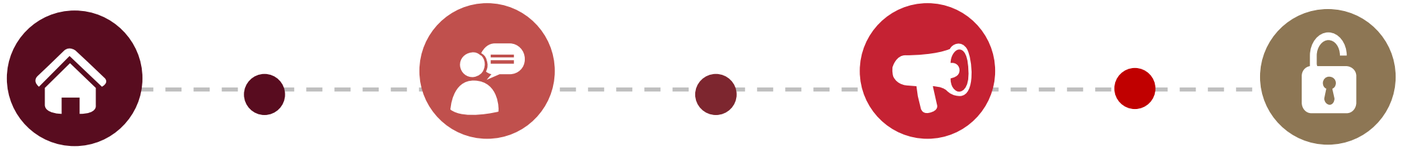
How a web server processes static web pages?



How a web server processes dynamic web pages?



Basic Web Development



What is web application?

- How web application works?
- Front-End and Back-End
- Code Editor

Basic Front-End

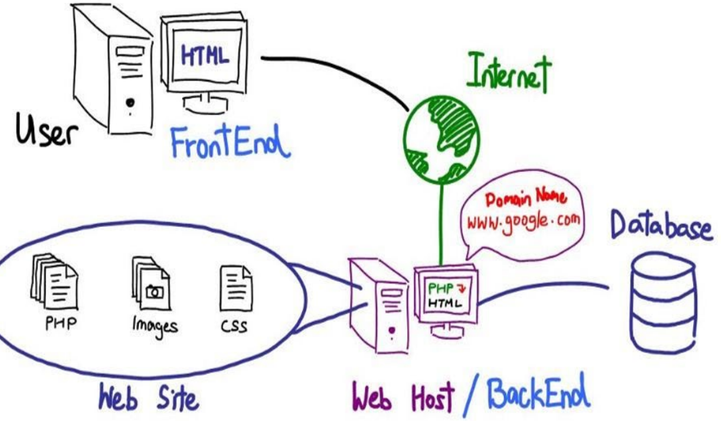
- HTML
- CSS
- Javascript

Tools

- Package Manager
- Build Tool
- Version Control

Basic Back-End

- Server
- Programming Language
- Database



Web application components

01 The browser provides the interface using HTML, CSS and JavaScript.

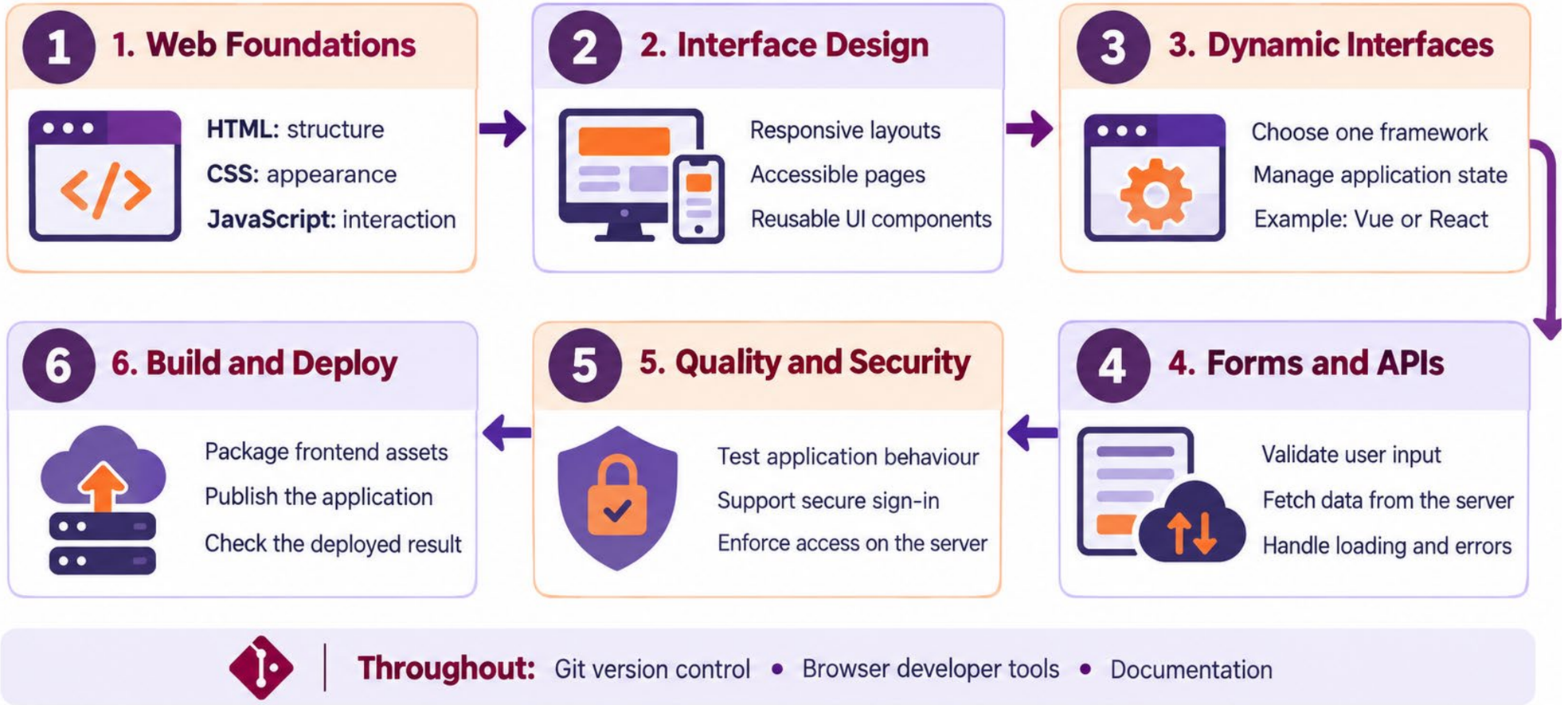
02 The server processes requests and applies application rules.

03 A database stores persistent data such as registrations.

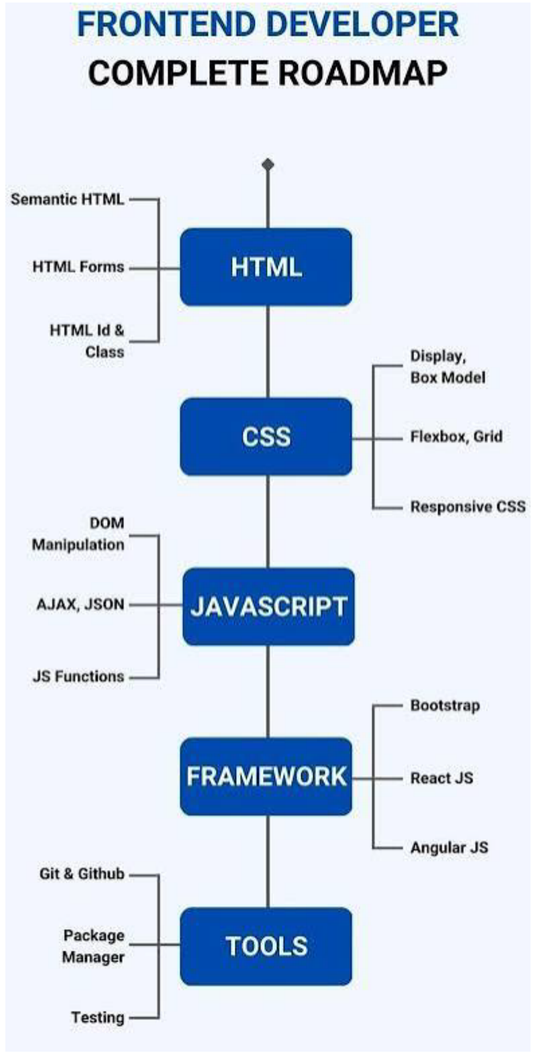
04 HTTP connects the client with the server. HTML describes page content.

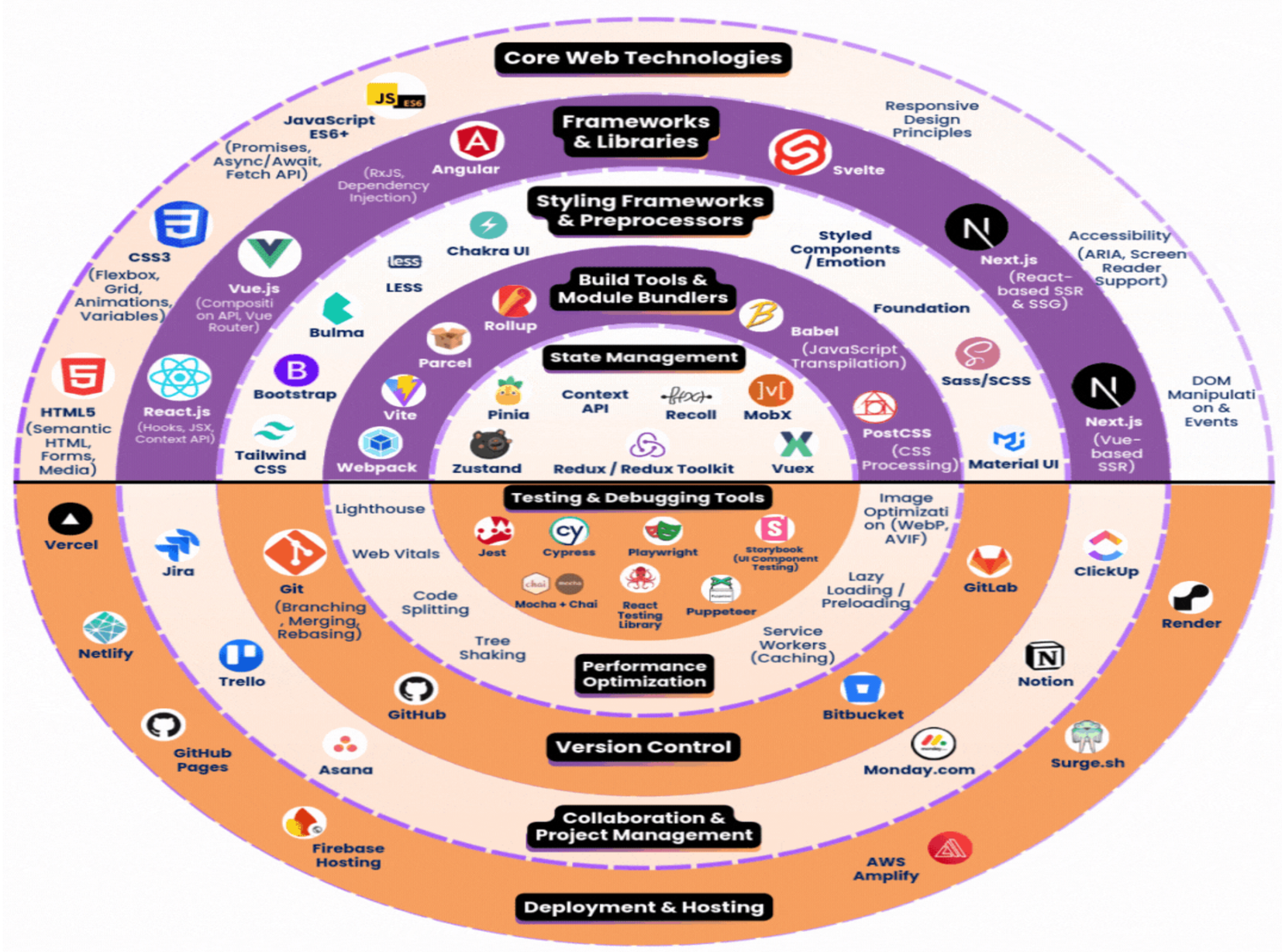
Frontend Development Roadmap

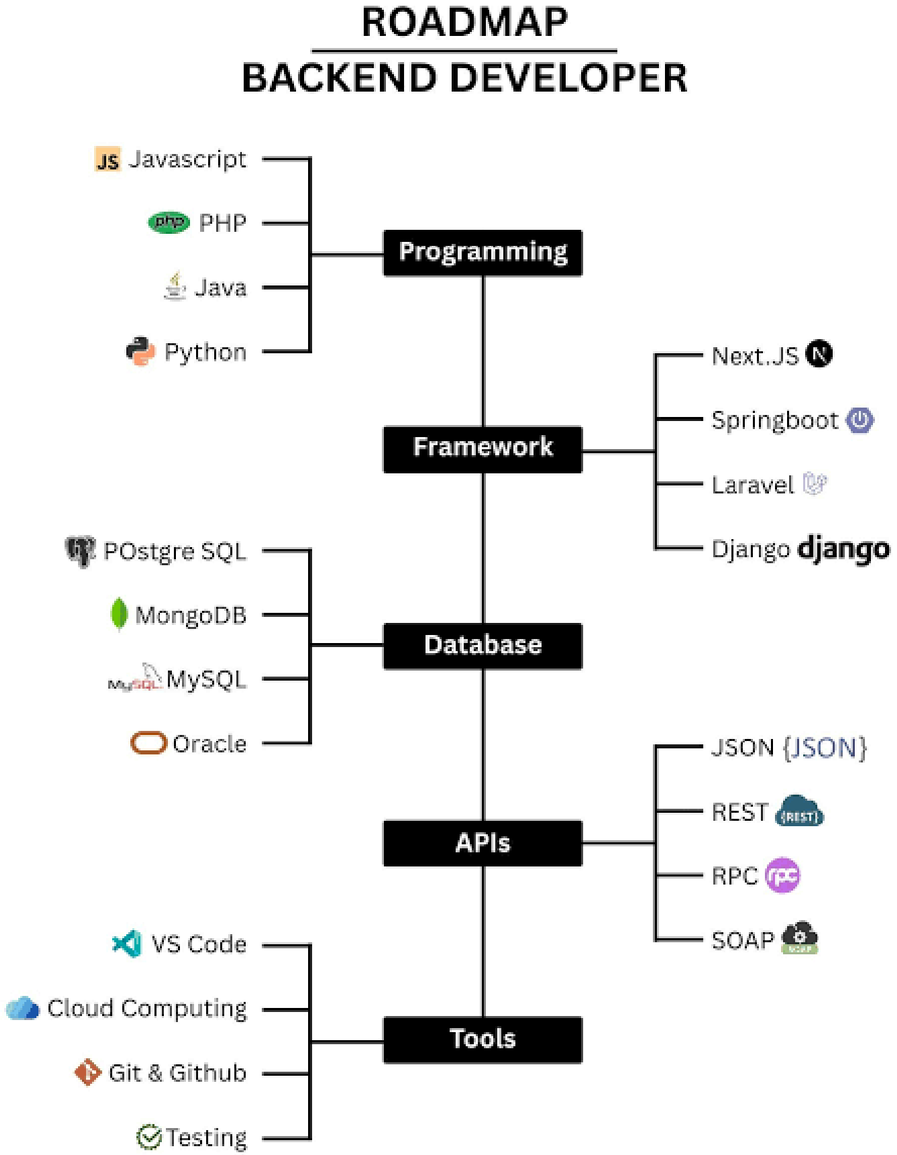
Learn the essentials first, then add tools as needed



Additional note: https://roadmap.sh/frontend

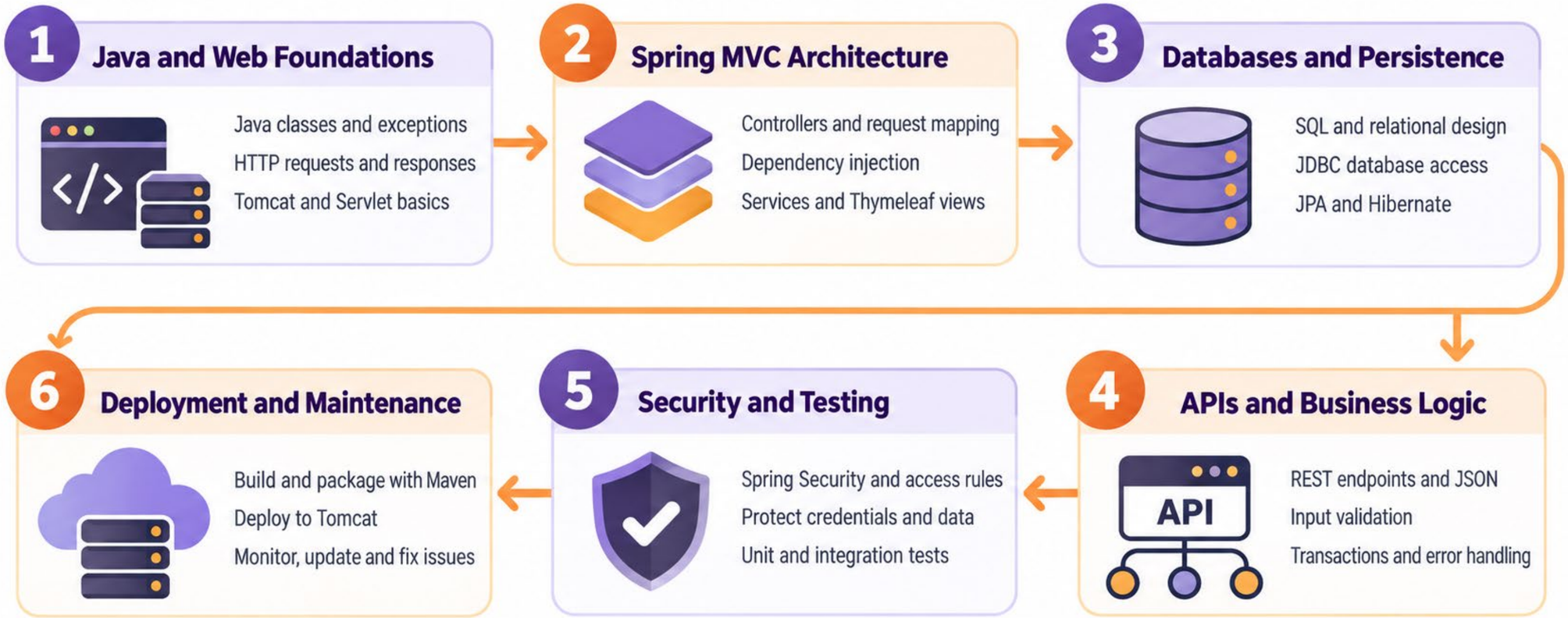

FRONTEND DEVELOPER
COMPLETE ROADMAP
Semantic HTML
HTML Forms
HTML Id & Class
HTML
Display, Box Model
Flexbox, Grid
Responsive CSS
CSS
DOM Manipulation
AJAX, JSON
JS Functions
JAVASCRIPT
Bootstrap
React JS
Angular JS
FRAMEWORK
Git & Github
Package Manager
Testing
TOOLS


Core Web Technologies
JavaScript ES6+ (Promises, Async/Await, Fetch API)
Frameworks & Libraries
Angular
(RxJS, Dependency Injection)
Svelte
Responsive Design Principles
CSS3 (Flexbox, Grid, Animations, Variables)
Styling Frameworks & Preprocessors
Chakra UI
Styled Components / Emotion
Next.js (React-based SSR & SSG)
Accessibility (ARIA, Screen Reader Support)
Vue.js (Compositi on API, Vue Router)
LESS
Build Tools & Module Bundlers
Rollup
Babel (JavaScript Transpilation)
Foundation
Bulma
Parcel
Sass/SCSS
HTML5 (Semantic HTML, Forms, Media)
React.js (Hooks, JSX, Context API)
Bootstrap
State Management
Vite
Pinia
Context API
Recoil
MobX
Next.js (Vue-based SSR)
DOM Manipulati on & Events
Tailwind CSS
Webpack
Zustand
Redux / Redux Toolkit
Vuex
PostCSS (CSS Processing)
Material UI
Testing & Debugging Tools
Lighthouse
Image Optimizati on (WebP, AVIF)
Vercel
Jira
Git (Branching , Merging, Rebasing)
Web Vitals
Jest
Cypress
Playwright
Storybook (UI Component Testing)
GitLab
ClickUp
Code Splitting
Mocha + Chai
React Testing Library
Puppeteer
Lazy Loading / Preloading
Render
Netlify
Tree Shaking
Service Workers (Caching)
Performance Optimization
Trello
Notion
GitHub
Bitbucket
Version Control
GitHub Pages
Asana
Monday.com
Surge.sh
Collaboration & Project Management
Firebase Hosting
AWS Amplify
Deployment & Hosting


ROADMAP
BACKEND DEVELOPER
Javascript
PHP
Java
Python
Programming
Next.JS
Springboot
Laravel
Django
Framework
POstgre SQL
MongoDB
MySQL
Oracle
Database
JSON {JSON}
REST
RPC
SOAP
APIs
VS Code
Cloud Computing
Git & Github
Testing
Tools

Backend Development Roadmap

A learning path for Java web applications



Additional note: https://roadmap.sh/backend

Java web evolution

01 Servlets + JSP: process requests and generate dynamic pages.

02 MVC: separate request handling, business logic and presentation.

03 Spring MVC: a Servlet-based framework for organised web development.

04 REST APIs: exchange data with browser or mobile clients.

Java web technologies and their roles

01 Servlet: Java API for processing HTTP requests and responses.

02 JSP: a server-side page technology that generates output such as HTML.

03 Spring MVC: organises web request handling around controllers and views.

04 Thymeleaf: renders HTML templates using data supplied by the application.

Java SE, Jakarta EE and Spring

01 Java SE provides the core Java platform. The JDK supports development.

02 Jakarta EE defines enterprise Java specifications, including Servlet.

03 Spring is a framework that integrates with selected Jakarta APIs.

04 Match the JDK, Spring and Tomcat versions before using example code.

A typical Java web application

01 Browser → HTTP → Tomcat / Spring MVC controller

02 Controller → service → repository → database

03 Response → browser: HTML, JSON or an error message

04 Separation of concerns makes behaviour easier to test and maintain.

Development tools at a glance

01 IDE or editor: write and debug Java code, for example STS or VS Code.

02 Maven: manage dependencies and automate the build through pom.xml.

03 Tomcat: run Servlet-based web applications. MySQL: store relational data.

04 Git: track code changes. GitHub: host repositories and support collaboration.

HTTP: request and response

01 A client sends a request; a server returns a response.

02 Request: method, path, headers and an optional body.

03 Response: status code, headers and an optional body.

04 HTTP is stateless; applications use sessions or tokens when needed.

Methods and status codes

01 GET retrieves data; POST submits data for processing.

02 PUT replaces a resource; PATCH updates part; DELETE removes it.

03 200 OK • 201 Created • 400 Bad Request • 404 Not Found

04 401 requires authentication; 403 denies access; 500 is a server error.

Hands-on: inspect an HTTP exchange

01 Open a demonstration page and the browser Network panel.

02 Reload; select a request and identify its method, path and status.

03 Compare request headers with response headers.

04 Explain what the exchange tells you—and what it cannot prove.

AI-assisted programming

01 AI supports code explanation, implementation, debugging and test drafting.

02 The developer reviews and understands suggestions before using them.

03 Check requirements, framework APIs and dependencies against documentation.

04 The developer retains responsibility for design decisions and code quality.

Vibe coding and developer oversight

01 Vibe coding uses conversational prompts to generate and modify software.

02 Developers iterate on prompts and observed results, sometimes with limited code inspection.

03 It overlaps with AI-assisted programming. The key issue is how much the developer understands and verifies.

04 Responsible use requires inspecting changes, testing behaviour and documenting assistance.

Vibe coding and AI-assisted programming

Aspect	Vibe coding	AI-assisted programming
Meaning	Conversational prompts generate and modify much of the code.	AI supports implementation, explanation, debugging and tests.
Typical approach	Iterate on prompts and results, sometimes with limited code inspection.	Review and understand suggestions before integrating them.
Responsibility	Verify the code before moving beyond experimentation.	Retain responsibility for design, testing and code quality.

The distinction is the level of code understanding and verification.

Why software engineering knowledge matters

01 Requirements and design: assess user needs and choose an appropriate architecture.

02 Verification: test correctness, security and failure cases.

03 Maintenance and ethics: keep code understandable, protect data and acknowledge reused work.

04 Example: a working registration page still needs validation, access checks and safe database queries.

Core software engineering and AI acceleration

Aspect	Core software engineering	AI-accelerated development
Approach	Engineers structure and implement the solution.	AI assists coding and repetitive tasks.
Iteration	Engineering work determines the delivery pace.	AI can support faster prototyping.
Testing	Manual and automated testing.	AI supports test generation and checks.
Ownership	Engineers control design and quality.	Engineers review outputs and retain responsibility.

Adapted from Qualhon Informatics (2026)

Project suitability

01 Core software engineering emphasis: complex integrations and security-sensitive platforms.

02 AI acceleration: prototypes, minimum viable products and internal automation.

03 Both approaches require architecture decisions, code review and testing.

04 Select the approach using project complexity and long-term needs.

Adapted from Qualhon Informatics (2026)

Application to this Java web course

01 Use AI to draft a small registration feature or suggest test cases.

02 Apply Spring MVC architecture and validate data on the server.

03 Review database access and test unauthorised requests.

04 Explain your changes and present evidence before approving the feature.

Privacy: minimise what you expose

01 Use synthetic data in examples, prompts and demonstrations.

02 Keep passwords, tokens and personal records out of AI prompts.

03 Collect only necessary data; restrict access and limit logging.

04 Review where data is sent, stored and retained.

Intellectual property: verify permission

01 Check licences before reusing code, images or datasets.

02 Publicly accessible content is not automatically free to reuse.

03 Record sources, licence terms and required attribution.

04 Review AI output for uncertain provenance or copied material.

Academic integrity: show your own learning

01 Follow the AI-use rules stated for each assessment.

02 Acknowledge assistance when required and identify its scope.

03 Explain your code, decisions, tests and limitations.

04 Present genuine evidence; never invent results or contributions.

Developer accountability

01 A working demo does not establish secure or reliable software.

02 Review input validation, authorisation and database access.

03 Test normal, invalid and unexpected inputs.

04 Report limitations honestly and respond constructively in Q&A.

Discuss: would you approve this application?

01 AI generates a student-registration application.

02 A real student dataset was included in the prompt.

03 The code works, but tests and licence checks were skipped.

04 Identify three risks; propose fixes and evidence of verification.

Recap

01 Trace one request through the Java web application stack.

02 Name one AI benefit and one verification step.

03 Explain one privacy or intellectual-property risk and its remedy.

04 Next: prepare the Java development environment.